



# ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

## ITERATIVE CONTROL STRUCTURES

**Professor:** *Gabriel E. Cabrera*  
**Teaching Assistant:** *Alex Den Braber*



### 1 *Loops* with & without `range()`

1. Build a list containing the names of the following technology companies: 'Facebook', 'Apple', 'Amazon' and 'Netflix'.
  - a. Print to the console each element in the list.
  - b. Print to the console each element in the list with its corresponding index.
  - c. Print to the console each letter that makes up each element (company) in the list.
2. Generate a sequence from 0 to 10 and print each value to the console.
3. Generate a sequence from 0 to 10 and print each value to the console in descending order.

### 2 *Conditionals*

1. Build the following sequence of numbers including the `None` values: 1, 2, `None`, 4, 5, 6, `None`. Then, using a *loop*, add up the elements that are numeric (do not consider `None`)
2. Build the following sequence of numbers including the `None` values: 1, 2, `None`, 4, 5, 6, `None`. Then, using a *loop*, add up all the numeric elements before the first `none`.
3. Build the following sequence of numbers: 1, 2, 3, 4, 5, 6, 7. Then, using a *loop*, add up the elements smaller than the number 5.
4. Add up all the numbers from 0 to 15 that are multiples of 3 or 5.
5. Build a *loop* that generates the Fibonacci sequence, which is characterized by each number being the sum of the two previous numbers. For example, the first 14 numbers of the sequence are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233.

### 3 *Comprehension*

A “*list comprehension*” allows you to build new lists (`list`) by filtering the elements of a collection (sequence) with a single expression. Its traditional form is the following:

```
[expr for val in collection if condition]
```

the above is equivalent to:

```
res = []  
for val in collection:
```

```

if condition:
    result.append(expr)

```

For example:

```
list2 = ['Facebook', 'Apple', 'Amazon', 'Netflix']
```

```
list2_title = [elem.upper() for elem in list2]
list2_title
```

```
## ['FACEBOOK', 'APPLE', 'AMAZON', 'NETFLIX']
```

Which in its standard form is equivalent to:

```
list2_title2 = []
```

```
for elem in list2:
    list2_title2.append(elem.upper())
```

```
list2_title2
```

```
## ['FACEBOOK', 'APPLE', 'AMAZON', 'NETFLIX']
```

You can even add the `else` conditional, which changes the structure of the *list comprehension* slightly, but the goal is the same:

```
[num if (num%3 == 0) or (num%5 == 0) else None for num in range(16)]
```

```
## [0, None, None, 3, None, 5, 6, None, None, 9, 10, None, 12, None, None, 15]
```

Which in its standard form is equivalent to:

```
res2 = []
```

```
for num in range(16):
    if (num%3 == 0) or (num%5 == 0):
        res2.append(num)
    else:
        res2.append(None)
```

```
res2
```

```
## [0, None, None, 3, None, 5, 6, None, None, 9, 10, None, 12, None, None, 15]
```

It can also be used to “flatten” nested lists:

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
[sub_elem for elem in nested_list for sub_elem in elem]
```

```
## [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Which in its standard form is equivalent to:

```
flattened_list = []
```

```
for elem in nested_list:
    for sub_elem in elem:
        flattened_list.append(sub_elem)
```

```
flattened_list
```

```
## [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

To see *comprehension* for dictionaries (`dict`) and tuples (`tuple`), see pages 67 and 68 of the book *Python for Data Analysis: Data Wrangling with Pandas, Numpy, and Ipython* by Wes McKinney.